INTRODUCTION TO ARTIFICIAL INTELLIGENCE · WEEK 1

Intelligent Agents

Rationality, PEAS and agent architectures, then Lab 1: build and benchmark your first agents.

2-hour session

Lecture + Lab + 24 quiz questions

```
f: P* -> A
PEAS = Performance, Environment, Actuators, Sensors
```

Dr.-Ing. Chukwuka Victor Obionwu · Miva Open University, School of Computing

PLAN · 2 HOURS

Session timeline

Part A · Lecture

Time	Segment	Min
0:00	Welcome & hook	5
0:05	Agents & agent function	8
0:13	Rationality & PEAS	10
0:23	Quiz 1	7
0:30	Agent architectures	13
0:43	Quiz 2	7
0:50	Break	5

Part B · Lab 1

Time	Segment	Min
0:55	Lab overview & PEAS	13
1:08	Environment & rewards	7
1:15	Quiz 3	8
1:23	Agents & simulation	17
1:40	Quiz 4	10
1:50	Recap & exit ticket	10

OBJECTIVES

By the end of today you can…

1. Define an agent, its percepts and its actions
2. Explain why rationality is not omniscience
3. Specify a task environment with PEAS and classify the four architectures
4. Build a reflex agent and design a model-based one in Lab 1

WARM-UP · 0:02

Which of these are agents?

- A thermostat
- A spam filter
- A robot vacuum

TURN TO A NEIGHBOUR · 2 MIN

For each one, name one thing it senses and one thing it does.

Perceive, then act

An agent is anything that perceives its environment through sensors and acts upon it through actuators.



The agent function

```
f: P* -> A
```

P* is the history of all percepts so far. A is the set of available actions.

AGENT FUNCTION
An abstract mapping from percept histories to actions.

AGENT PROGRAM
The concrete implementation running on real or virtual hardware.

Percept sequence (vacuum agent)	Action
[(0,0) dirty]	suck
[(0,0) dirty], [(0,0) clean]	move_south
… [(0,1) dirty]	suck

What makes an action rational?

FOUR FACTORS

1. The performance measure defining success
2. The agent's prior knowledge of the environment
3. The actions the agent can perform
4. The agent's percept sequence to date

RATIONAL

Maximizes **expected** performance given current knowledge.

OMNISCIENT

Knows the **actual** outcome of its actions. Impossible in reality.

DISCUSS · 2 MIN

A drone takes the safest known route and a freak gust still knocks it off course. Was it irrational?

Specify the task environment first

P — **Performance measure**
Objective metrics used to evaluate the agent's behavior

E — **Environment**
External conditions, constraints and state space

A — **Actuators**
Mechanisms through which the agent acts on the environment

S — **Sensors**
Input devices or APIs through which percepts arrive

> Software architects define PEAS **before** designing the agent.

WORKED EXAMPLE · 0:20

PEAS: autonomous delivery agent

Component	Description
Performance	Safety, on-time delivery percentage, energy efficiency, customer satisfaction
Environment	Urban streets, sidewalks, weather conditions, dynamic pedestrian obstacles
Actuators	Electric drive motors, steering servos, indicator lights, notification speaker
Sensors	Cameras, LiDAR, ultrasonic proximity sensors, GPS module, IMU telemetry

THINK · 1 MIN

Safety and energy efficiency can conflict. Who decides the trade-off? Hold that thought for utility-based agents.

QUIZ 1 · 0:23

Questions 1–3

QUESTION 1

What is an agent?

- **A** Software that only stores data
- **B** Anything that senses and acts on its environment
- **C** A human using an AI tool
- **D** A database of rules

QUESTION 2

The agent function f: P* -> A maps…

- **A** actions to percepts
- **B** one sensor to one actuator
- **C** percept histories to actions
- **D** goals to utilities

QUESTION 3

Which is NOT a factor in rationality?

- **A** The performance measure
- **B** Prior knowledge
- **C** The percept sequence to date
- **D** Knowing every outcome in advance

Vote by show of hands, then reveal on the next slide.

QUIZ 1 · ANSWERS

Answers 1–3

QUESTION 1

What is an agent?

Answer: B

Anything that senses and acts on its environment

It perceives through sensors and acts through actuators. That is the whole definition.

QUESTION 2

The agent function f: P* -> A maps…

Answer: C

percept histories to actions

P* is the whole history of percepts, not just the latest one.

QUESTION 3

Which is NOT a factor in rationality?

Answer: D

Knowing every outcome in advance

That is omniscience. The four factors are performance measure, prior knowledge, available actions and percept sequence.

QUIZ 1 · 0:27

Questions 4–6

QUESTION 4

True or false: a rational agent always gets the best actual outcome.

- **A** True
- **B** False

QUESTION 5

Rationality maximizes expected performance. Omniscience maximizes…

- **A** actual performance
- **B** average performance
- **C** worst-case performance
- **D** computing speed

QUESTION 6

In the delivery agent's PEAS, LiDAR and GPS are…

- **A** Performance measures
- **B** Actuators
- **C** Sensors
- **D** Environment

Vote by show of hands, then reveal on the next slide.

QUIZ 1 · ANSWERS

Answers 4–6

QUESTION 4

A rational agent always gets the best actual outcome.

Answer: B

False

Rationality maximizes expected performance. Luck or hidden information can still make the actual outcome worse.

QUESTION 5

Omniscience maximizes…

Answer: A

actual performance

An omniscient agent knows the real outcome of each action. In reality that is impossible.

QUESTION 6

LiDAR and GPS in the delivery PEAS are…

Answer: C

Sensors

Cameras, LiDAR, ultrasonic sensors, GPS and IMU telemetry are how the agent receives percepts.

Four architectures, more deliberation each

1

Simple reflex

Acts purely on the current percept. Condition-action rules, no history.

2

Model-based reflex

Keeps internal state to track unseen parts of the world.

3

Goal-based

Combines state tracking with explicit goals to plan actions.

4

Utility-based

Uses a utility function to trade off competing goals.

Lab 1 builds architecture **1** and then upgrades it to architecture **2**.

LECTURE CODE · 0:34

A simple reflex agent in Python

```
class SimpleReflexAgent:
    def __init__(self) -> None:
        self.rules = {
            "obstacle_ahead": "stop_and_turn",
            "clear_path": "move_forward",
            "battery_low": "return_to_dock",
        }

    def select_action(self, percept):
        status = percept.data.get("status", "clear_path")
        return self.rules.get(status, "move_forward")
```

PREDICT · 1 MIN

What does the agent do when the status is "fog"?

NOTICE

No memory. The same percept always gives the same action.

LECTURE CODE · THE CONTROL LOOP · 0:37

Sense, decide, act, repeat

```
def agent_loop(agent, environment):
    while environment.is_active():
        percept = environment.get_percept()
        action = agent.select_action(percept)
        environment.apply_action(action)
```

1 Sense: get_percept()

2 Decide: select_action()

3 Act: apply_action()

PREDICT · 1 MIN

What ends this loop? Where would a performance score be tracked?

ACTIVITY · 0:40

Which architecture fits?

1. A thermostat switches the heater on when the temperature drops below 18 degrees.
2. A robot vacuum remembers which rooms it has already cleaned.
3. A navigation app plans a route to a destination you typed in.
4. A trading agent balances expected return against risk.

Simple reflex **Model-based** **Goal-based** **Utility-based**

QUIZ 2 · 0:43

Questions 1–3

QUESTION 1

Which architecture acts only on the current percept, using condition-action rules?

- **A** Utility-based
- **B** Goal-based
- **C** Model-based reflex
- **D** Simple reflex

QUESTION 2

What does a model-based reflex agent add to a simple reflex agent?

- **A** Internal state tracking unseen parts of the world
- **B** A utility function
- **C** Explicit goal destinations
- **D** Faster sensors

QUESTION 3

Which architecture uses a utility function to trade off competing goals?

- **A** Goal-based
- **B** Utility-based
- **C** Simple reflex
- **D** Model-based reflex

Vote by show of hands, then reveal on the next slide.

QUIZ 2 · ANSWERS

Answers 1–3

QUESTION 1

Acts only on the current percept, using condition-action rules?

Answer: D

Simple reflex

It ignores history entirely: same percept in, same action out.

QUESTION 2

What a model-based reflex agent adds?

Answer: A

Internal state tracking unseen parts of the world

Goals are the next step up, and utilities the one after that.

QUESTION 3

Uses a utility function to trade off goals?

Answer: B

Utility-based

A utility function scores outcomes, so competing goals can be traded off.

QUIZ 2 · 0:47

Questions 4–6

QUESTION 4

True or false: in the lecture’s SimpleReflexAgent, an unknown status such as "fog" leads to "move_forward".

- **A** True
- **B** False

QUESTION 5

With status "battery_low", the lecture’s agent chooses…

- **A** stop_and_turn
- **B** move_forward
- **C** return_to_dock
- **D** noop

QUESTION 6

An agent that weighs speed against safety when choosing a route is…

- **A** Simple reflex
- **B** Utility-based
- **C** Model-based reflex
- **D** Not an agent

Vote by show of hands, then reveal on the next slide.

QUIZ 2 · ANSWERS

Answers 4–6

QUESTION 4

Unknown status "fog" leads to "move_forward".

Answer: A

True

rules.get(status, "move_forward") returns the default when no rule matches.

QUESTION 5

Status "battery_low" gives…

Answer: C

return_to_dock

It is one of the three condition-action rules in the dictionary.

QUESTION 6

Weighs speed against safety…

Answer: B

Utility-based

Trading off competing goals is exactly what a utility function is for.

BREAK · 0:50–0:55

Five minutes. Back at 0:55.

BEFORE YOU RETURN

Open the starter notebook and check that Python 3 runs.

Next: Lab 1, Building PEAS Agent Architectures.

LAB 1 · OVERVIEW · 0:55

Building PEAS agent architectures

1

PEAS analysis

Write a complete PEAS analysis for an AI Automated Code Reviewer.

2

Model-based agent

Extend SimpleReflexiveAgent into a ModelBasedAgent with an internal map of clean and dirty cells.

3

Benchmark

Run 100 trials and compare the average score of the two agents.

SUBMISSION

Your code must pass every unit test in test_lab1.py before you open a pull request on the course grading repo.

LAB 1 · PEAS SPECIFICATIONS · 0:59

Four agent domains

Agent	Performance	Environment	Actuators	Sensors
Vacuum cleaner	Cleanliness, energy, time	Grid cells, dirt, walls	Motors, suction, steering	Bump, dirt, location
Taxi driver	Safe arrival, speed, legality, comfort	Roads, traffic, pedestrians, weather	Steering, accelerator, brake, horn	Cameras, LiDAR, radar, GPS
Medical assistant	Diagnostic accuracy, patient outcome	Patient records, lab tests, symptoms	Reports, treatment plans	Keyboard input, EHR feeds
Trading agent	ROI, risk-adjusted returns, low slippage	Exchange feeds, news, order books	Limit, market, cancel orders	Price feeds, order depth API

> The vacuum row is the one our simulation implements.

GROUP ACTIVITY · LAB PART 1 · 1:03

PEAS for an AI automated code reviewer

P

How do we measure a good reviewer?

E

What world does it work in?

A

What can it do to that world?

S

What can it observe?

GROUPS OF 3 · 5 MIN

Write 2 to 3 items per letter. Then swap with another group and look for duplicates between boxes.

LAB CODE · THE ENVIRONMENT · 1:08

The simulated grid world

```
class Environment:
    def __init__(self, width=5, height=5): ...

    def reset(self):
        # 30% chance of dirt in each cell
        grid[(x, y)] = "dirty" if random.random() < 0.3
else "clean"

    def get_percept(self, agent_location):
        return {"location": ..., "status": ...}

    def execute_action(self, agent_location, action):
        # returns (new_location, reward)
```

DESIGN STRATEGY

The environment makes percepts. The agent returns actions. Keep the agent's internal state apart from the full ground truth.

ACTIONS

`suck`, `move_north`, `move_south`, `move_east`, `move_west`, `noop`

LAB CODE · THE PERFORMANCE MEASURE · 1:12

How the environment scores actions

Action	Effect	Reward
suck on a dirty cell	Cell becomes clean	+10
suck on a clean cell	Nothing changes	−1
move_north / south / east / west	Move one cell, stopping at walls	−1
noop	Nothing happens	0

THINK · 2 MIN

A move into a wall still costs −1. What behavior does this reward design encourage or discourage?

QUIZ 3 · 1:15

Questions 1–3

QUESTION 1

The agent stands on a dirty cell and chooses “suck”. The reward is…

- **A** 0
- **B** −1
- **C** +10
- **D** +1

QUESTION 2

The agent stands on a clean cell and chooses “suck”. The reward is…

- **A** −1
- **B** 0
- **C** +10
- **D** +5

QUESTION 3

At (0, 0) the agent chooses move_north. What happens?

- **A** Moves to (0, −1), reward −1
- **B** Stays at (0, 0), reward 0
- **C** Moves to (0, 1), reward −1
- **D** Stays at (0, 0), reward −1

Vote by show of hands, then reveal on the next slide.

QUIZ 3 · ANSWERS

Answers 1–3

QUESTION 1

Suck on a dirty cell: reward?

Answer: C

+10

The cell becomes clean and the agent earns the cleaning reward.

QUESTION 2

Suck on a clean cell: reward?

Answer: A

−1

An unnecessary suck is penalized.

QUESTION 3

move_north at (0, 0)?

Answer: D

Stays at (0, 0), reward −1

y = max(0, y − 1) stops at the wall, but every move costs energy.

QUIZ 3 · 1:19

Questions 4–6

QUESTION 4

True or false: in the starter code the environment generates the percept and the agent returns an action.

- **A** True
- **B** False

QUESTION 5

By default each cell starts dirty with probability…

- **A** 10%
- **B** 30%
- **C** 50%
- **D** 70%

QUESTION 6

Expected number of dirty cells in a 3 x 3 grid after reset?

- **A** 0.3
- **B** 1.5
- **C** 2.7
- **D** 9

Vote by show of hands, then reveal on the next slide.

QUIZ 3 · ANSWERS

Answers 4–6

QUESTION 4

Environment generates percepts; agent returns actions.

Answer: A

True

get_percept belongs to Environment and perceive_and_act returns the action.

QUESTION 5

Default dirt probability per cell?

Answer: B

30%

reset() uses random.random() < 0.3.

QUESTION 6

Expected dirty cells in 3 x 3?

Answer: C

2.7

9 cells times 0.3 gives 2.7 on average. A 5 x 5 grid gives 7.5.

LAB CODE · THE REFLEX AGENT · 1:23

SimpleReflexiveAgent

```
def perceive_and_act(self, percept):
    status = percept.get("status")
    if status == "dirty":
        return "suck"

    # Simple random exploration rule
    actions = ["move_north", "move_south",
               "move_east", "move_west"]
    return random.choice(actions)
```

TWO RULES

1. Dirty here: suck.
2. Otherwise: random move.

THINK · 2 MIN

Does this agent ever know the grid is clean? What does it cost it?

LAB SIMULATION · ONE SAMPLE RUN · 1:26

Reading a trace of the reflex agent

Step	Location, status	Action	Score
01	(0,0) dirty	suck	10
02	(0,0) clean	move_south	9
03	(0,1) dirty	suck	19
04	(0,1) clean	move_east	18
05	(1,1) dirty	suck	28
06	(1,1) clean	move_east	27
07	(2,1) clean	move_south	26
08	(2,2) clean	move_north	25
09	(2,1) clean	move_east	24
10	(2,1) clean	move_south	23

Step 09: the agent is at the east wall, so it stays put and still pays −1.

Final score 23: three cleanings (+30) and seven moves (−7).

Steps 07 to 08 wander back and forth over clean cells.

LAB PART 2 · DESIGN DISCUSSION · 1:30

Designing the ModelBasedAgent

1 REMEMBER

An internal map of what it has seen: each known cell is clean or dirty.

```
(x, y) -> "clean" | "dirty"
```

2 UPDATE

Every step, write the current percept into the map. After sucking, mark that cell clean.

3 DECIDE

Dirty here: suck. Otherwise move toward a neighbor not yet known to be clean.

EDGE CASES · 3 MIN

What should it do when every neighbor is known clean? How can it learn where the walls are?

LAB PART 3 · BENCHMARK PLAN · 1:35

100 trials, one average per agent

```
# sketch, not the graded solution
scores = []
for trial in range(100):
    env = Environment(3, 3)
    agent = SimpleReflexiveAgent()
    # run the same number of steps
    scores.append(agent.total_score)

average = sum(scores) / len(scores)
```

FAIR COMPARISON

Same grid size, same step count, and ideally the same dirt layouts for both agents.

PREDICT · 1 MIN

Which agent will score higher, and by how much?

QUIZ 4 · 1:40

Questions 1–3

QUESTION 1

What turns SimpleReflexiveAgent into a ModelBasedAgent?

- **A** An internal map of clean and dirty cells
- **B** A faster random generator
- **C** A bigger grid
- **D** A different reward function

QUESTION 2

Why does the lab ask for 100 benchmark trials?

- **A** To make the grid bigger
- **B** Python needs a warm-up
- **C** To avoid writing unit tests
- **D** Dirt and moves are random, so averaging reduces noise

QUESTION 3

For an AI code reviewer, which item is an ACTUATOR?

- **A** Bug-detection rate
- **B** Posting review comments on the pull request
- **C** The repository and its history
- **D** The diff and CI results

Vote by show of hands, then reveal on the next slide.

QUIZ 4 · ANSWERS

Answers 1–3

QUESTION 1

What makes the agent model-based?

Answer: A

An internal map of clean and dirty cells

Remembering past percepts is exactly what a simple reflex agent lacks.

QUESTION 2

Why 100 benchmark trials?

Answer: D

Averaging reduces noise

Random dirt and random moves make any single run unreliable.

QUESTION 3

Which is an actuator?

Answer: B

Posting review comments

A is performance, C is environment, D is sensors.

QUIZ 4 · 1:45

Questions 4–6

QUESTION 4

True or false: the model-based agent should read the environment's full grid directly.

- **A** True
- **B** False

QUESTION 5

A cell is marked clean on the agent's internal map. The agent should…

- **A** Suck there again
- **B** Mark it dirty
- **C** Avoid revisiting it when choosing where to go
- **D** Stop the simulation

QUESTION 6

In Part 3, the two agents are compared by…

- **A** Average score over 100 trials
- **B** Best single run
- **C** Number of grid cells
- **D** Lines of code

Vote by show of hands, then reveal on the next slide.

QUIZ 4 · ANSWERS

Answers 4–6

QUESTION 4

The agent reads the environment’s full grid directly.

Answer: B

False

It must build its map from percepts. Reading the ground truth would bypass the sensors.

QUESTION 5

A cell is marked clean on the map.

Answer: C

Avoid revisiting it

Skipping known-clean cells is the point of the model: fewer wasted moves.

QUESTION 6

Part 3 compares agents by…

Answer: A

Average score over 100 trials

Averages smooth out the randomness that single runs suffer from.

RECAP · 1:50

Four ideas to keep

Agents: sensors in, actuators out, behavior given by f: P* -> A.

Rationality: maximize expected performance. That is not omniscience.

PEAS first, then pick an architecture: reflex, model-based, goal-based or utility-based.

Lab 1: rewards define performance, and memory should beat random wandering.

EXIT TICKET · 5 MIN · WRITE ON PAPER

1. Name the four PEAS components.
2. Which architecture needs internal state?
3. An agent sucks a dirty cell, moves three times, then sucks a clean cell. Its score?